SystemVerilog and UVM Practice Journey

Phase 03 — Functions, Tasks & Packages

Bibhu Asish Panda

November 10, 2026

Contents

3 Phase 03 — Functions, Tasks & Packages 1
- **3.1 Overview** 1
- **3.2 Topics Covered** 1
  - **3.2.1 Functions, Tasks and Reusability** 1
  - **3.2.2 Packages and Shared Definitions** 1
- **3.3 Key Learning Areas** 2
- **3.4 Practical Work** 2
- **3.5 Learning Outcomes** 2
- **3.6 Tools** 2
- **3.7 SV-025 — Functions** 3
  - **3.7.1 Function Declaration** 3
  - **3.7.2 Function Arguments** 3
  - **3.7.3 Return Value** 3
  - **3.7.4 Verification Relevance** 4
  - **3.7.5 Complete Runnable Code** 4
  - **3.7.6 Output** 4
- **3.8 SV-026 — Tasks** 5
  - **3.8.1 Task Declaration** 5
  - **3.8.2 Timing Controls** 5
  - **3.8.3 Verification Relevance** 5
  - **3.8.4 Complete Runnable Code** 6
  - **3.8.5 Output** 6
- **3.9 SV-027 — Function/Task Arguments** 7
  - **3.9.1 `input`** 7
  - **3.9.2 `output`** 7
  - **3.9.3 `inout`** 7
  - **3.9.4 `ref`** 7
  - **3.9.5 Verification Relevance** 8
  - **3.9.6 Complete Runnable Code** 8
  - **3.9.7 Output** 9
- **3.10 SV-028 — Lifetime** 10
  - **3.10.1 Static Lifetime** 10
  - **3.10.2 Automatic Lifetime** 10
  - **3.10.3 Static vs Automatic** 10
  - **3.10.4 Verification Relevance** 11
  - **3.10.5 Complete Runnable Code** 11
  - **3.10.6 Output** 11

3.11 SV-029 — Void Functions **13**
3.11.1 `void` Return Type **13**
3.11.2 `void'(expression)` **13**
3.11.3 Difference **13**
3.11.4 Verification Relevance **13**
3.11.5 Complete Runnable Code **14**
3.11.6 Output **14**
3.12 SV-030 — Packages **15**
3.12.1 `package` **15**
3.12.2 `import` **15**
3.12.3 Scope Resolution `::` **15**
3.12.4 Verification Relevance **16**
3.12.5 Complete Runnable Code **16**
3.12.6 Output **17**
3.13 SV-031 — Package + Types + Classes **18**
3.13.1 Typedefs in a Package **18**
3.13.2 Enums in a Package **18**
3.13.3 Classes in a Package **18**
3.13.4 Package Functions **18**
3.13.5 Verification Relevance **19**
3.13.6 Complete Runnable Code **19**
3.13.7 Output **20**

Chapter 3

Phase 03 — Functions, Tasks & Packages

3.1 Overview

This phase covers reusable code blocks and shared definitions in SystemVerilog. It introduces functions and tasks, argument passing, variable lifetime, and packages for organizing and reusing types, classes, and functions across a project.

3.2 Topics Covered

3.2.1 Functions, Tasks and Reusability

No.	Topic	Concepts Covered
SV-025	Functions	Function declaration, arguments, return value
SV-026	Tasks	Task declaration, timing controls
SV-027	Function/Task Arguments	`input`, `output`, `inout`, `ref`
SV-028	Lifetime	Automatic lifetime, static lifetime
SV-029	Void Functions	`void` return type, `void'(expression)`

3.2.2 Packages and Shared Definitions

No.	Topic	Concepts Covered
SV-030	Packages	`package`, `import`, scope resolution `::`
SV-031	Package + Types + Classes	Typedefs, enums, classes, package functions

3.3 Key Learning Areas

- **Reusable code:** Create functions and tasks to organize repeated operations.
- **Argument passing:** Understand how `input`, `output`, `inout`, and `ref` affect data transfer.
- **Execution behavior:** Distinguish automatic and static lifetime and understand timing controls in tasks.
- **Return handling:** Use `void` functions and `void'(expression)` appropriately.
- **Code organization:** Use packages, imports, and scope resolution to share definitions.
- **Reusable definitions:** Group typedefs, enums, classes, and functions into packages.

3.4 Practical Work

Each topic is practiced through runnable SystemVerilog examples and Vivado simulation to observe execution behavior, argument handling, and the reuse of package definitions.

3.5 Learning Outcomes

After completing this phase, you should be able to: - Write and call functions and tasks. - Pass arguments using different directions and references. - Explain static and automatic variable lifetime. - Use void-returning operations appropriately. - Organize shared types, classes, and functions in packages.

3.6 Tools

- **Language:** SystemVerilog
- **Simulator:** AMD Vivado
- **Practice:** Runnable code examples, expected outputs, and GitHub learning notes

3.7 SV-025 — Functions

A **function** is a reusable block of SystemVerilog code that performs a specific operation and returns a value.

Functions help reduce repeated code and make verification code easier to organize.

3.7.1 Function Declaration

Basic syntax:

```
function return_type function_name(input type argument);
    // function body
    return value;
endfunction
```

Example:

```
function int add_numbers(input int a, input int b);
    return a + b;
endfunction
```

Here:

- `function` declares the function.
- `int` is the return type.
- `add_numbers` is the function name.
- `a` and `b` are input arguments.
- `return` provides the function result.

3.7.2 Function Arguments

Arguments allow values to be passed into a function.

```
add_numbers(10, 20);
```

The value `10` is passed to `a`, and `20` is passed to `b`.

3.7.3 Return Value

The function returns the result using:

```
return a + b;
```

The returned value can be assigned to a variable:

```
result = add_numbers(10, 20);
```

It can also be used directly in an expression:

```
$display("%0d", add_numbers(5, 15));
```

3.7.4 Verification Relevance

Functions are commonly used in verification for:

- Data calculations
- Reference-model operations
- Comparisons and checks
- Reusable testbench utilities
- Address/data transformations
- Protocol-related helper operations

3.7.5 Complete Runnable Code

```
module sv_025_functions;

    // Function declaration with two arguments and an integer return value
    function int add_numbers(input int a, input int b);
        return a + b;
    endfunction

    int result;

    initial begin
        result = add_numbers(10, 20);

        $display("10 + 20 = %0d", result);
        $display("5 + 15  = %0d", add_numbers(5, 15));
    end

endmodule
```

3.7.6 Output

```
10 + 20 = 30
5 + 15  = 20
```

3.8 SV-026 — Tasks

A **task** is a reusable block of SystemVerilog code that can perform multiple operations and can contain timing controls.

Unlike functions, tasks can consume simulation time using controls such as `#`, `@`, and `wait`.

3.8.1 Task Declaration

Basic syntax:

```
task task_name(input type argument);
    // task body
endtask
```

Example:

```
task automatic print_message(input string message);
    #10;
    $display("[%0t] %s", $time, message);
endtask
```

The `automatic` keyword gives each task invocation its own storage, which is useful when multiple task calls may execute independently.

3.8.2 Timing Controls

Tasks can contain timing controls such as:

```
#10;
```

This advances simulation time by 10 time units.

Other common timing controls include:

```
@(posedge clk);
wait (done);
```

These allow tasks to model activities that depend on simulation events.

3.8.3 Verification Relevance

Tasks are commonly used in verification for:

- Driving stimulus
- Waiting for clock edges
- Applying transactions
- Reset sequences
- Protocol operations
- Reusable testbench procedures

For example, a testbench task can drive a complete transaction and wait for the DUT to respond.

3.8.4 Complete Runnable Code

```
module sv_026_tasks;

    // Task with an input argument and timing control
    task automatic print_message(input string message);
        #10;
        $display("[%0t] %s", $time, message);
    endtask

    initial begin
        $display("[%0t] Start", $time);

        print_message("First task call");
        print_message("Second task call");

        $display("[%0t] End", $time);
    end

endmodule
```

3.8.5 Output

```
[0] Start
[10000] First task call
[20000] Second task call
[20000] End
```

3.9 SV-027 — Function/Task Arguments

SystemVerilog provides different argument modes for functions and tasks. The main modes are `input`, `output`, `inout`, and `ref`.

3.9.1 input

`input` passes a value into the function or task.

```
task automatic show_input(input int value);
    $display("value = %0d", value);
endtask
```

The procedure can read the value but does not use it to modify the caller's variable.

3.9.2 output

`output` is used to return a value from a function or task.

```
task automatic calculate_sum(
    input int a,
    input int b,
    output int result
);
    result = a + b;
endtask
```

The caller receives the value assigned to `result`.

3.9.3 inout

`inout` allows the procedure to read and modify the argument.

```
task automatic increment_inout(inout int value);
    value = value + 1;
endtask
```

After the task completes, the caller sees the modified value.

3.9.4 ref

`ref` passes a reference to the original variable.

```
task automatic increment_ref(ref int value);
    value = value + 10;
endtask
```

The task directly operates on the caller's variable.

3.9.5 Verification Relevance

Argument modes are important when creating reusable verification components and procedures. They are commonly used to:

- Pass stimulus into tasks
- Return calculated or checked results
- Modify testbench data
- Operate directly on transaction objects or variables
- Build reusable verification utilities

3.9.6 Complete Runnable Code

```
module sv_027_function_task_arguments;

    // input: value is passed into the task
    task automatic show_input(input int value);
        $display("input  value  = %0d", value);
    endtask

    // output: task produces a result for the caller
    task automatic calculate_sum(
        input  int a,
        input  int b,
        output int result
    );
        result = a + b;
    endtask

    // inout: task can read and modify the argument
    task automatic increment_inout(inout int value);
        value = value + 1;
    endtask

    // ref: task directly accesses the caller's variable
    task automatic increment_ref(ref int value);
        value = value + 10;
    endtask

    int result;
    int data;

    initial begin

        show_input(25);

        calculate_sum(10, 20, result);
        $display("output result = %0d", result);

        data = 5;
        increment_inout(data);
```

```
        $display("inout data     = %0d", data);

        data = 5;
        increment_ref(data);
        $display("ref data       = %0d", data);

    end

endmodule
```

3.9.7 Output

```
input  value  = 25
output result = 30
inout data     = 6
ref data       = 15
```

3.10 SV-028 — Lifetime

Lifetime determines how long a variable's storage exists and how its value behaves across procedure calls.

SystemVerilog supports **static** and **automatic** lifetimes.

3.10.1 Static Lifetime

A variable with static lifetime has one storage location that persists throughout the simulation.

```
function int static_count();
    static int count = 0;
    count++;
    return count;
endfunction
```

Each call uses the same `count` variable:

```
Call 1 -> 1
Call 2 -> 2
Call 3 -> 3
```

The value is preserved between calls.

3.10.2 Automatic Lifetime

An automatic procedure creates a new storage instance for each activation.

```
function automatic int automatic_count();
    int count = 0;
    count++;
    return count;
endfunction
```

Each call gets a new `count`, initialized to `0`:

```
Call 1 -> 1
Call 2 -> 1
Call 3 -> 1
```

Automatic lifetime is particularly useful when multiple procedure calls need independent local variables.

3.10.3 Static vs Automatic

Lifetime	Storage	Value between calls
Static	One shared instance	Preserved
Automatic	New instance per activation	Not preserved

Tasks and functions can use automatic lifetime, which is especially useful for reusable verification procedures and concurrent operations.

3.10.4 Verification Relevance

Lifetime becomes important when writing:

- Reusable tasks and functions
- Concurrent testbench operations
- Recursive procedures
- Independent procedure calls
- Verification utility code

Using automatic lifetime helps prevent different procedure activations from unintentionally sharing local variables.

3.10.5 Complete Runnable Code

```
module sv_028_lifetime;

    // Static lifetime: one variable is shared by all calls
    function int static_count();
        static int count = 0;
        count++;
        return count;
    endfunction

    // Automatic lifetime: new variable for every call
    function automatic int automatic_count();
        int count = 0;
        count++;
        return count;
    endfunction

    initial begin
        $display("Static lifetime:");
        $display("Call 1 = %0d", static_count());
        $display("Call 2 = %0d", static_count());
        $display("Call 3 = %0d", static_count());

        $display("\nAutomatic lifetime:");
        $display("Call 1 = %0d", automatic_count());
        $display("Call 2 = %0d", automatic_count());
        $display("Call 3 = %0d", automatic_count());
    end

endmodule
```

3.10.6 Output

```
Static lifetime:
Call 1 = 1
Call 2 = 2
```

```
Call 3 = 3

Automatic lifetime:
Call 1 = 1
Call 2 = 1
Call 3 = 1
```

3.11 SV-029 — Void Functions

A **void function** performs an operation without returning a value.

SystemVerilog also provides `void'(expression)` to explicitly evaluate an expression and discard its return value.

3.11.1 `void` Return Type

A function can be declared with `void` as its return type:

```
function void print_sum(input int a, input int b);
    $display("Sum = %0d", a + b);
endfunction
```

The function performs an operation but does not return a value to the caller.

It can be called normally:

```
print_sum(10, 20);
```

3.11.2 `void'(expression)`

`void'(expression)` is a casting expression used when the result of an expression is intentionally not needed.

For example:

```
void'(multiply(7, 3));
```

The `multiply()` function still returns its integer result, but the result is explicitly discarded.

This is different from declaring the function itself as `void`.

3.11.3 Difference

Syntax	Meaning
`function void ...`	Function has no return value
`void'(expression)`	Evaluate expression and discard its result

3.11.4 Verification Relevance

Void functions are useful for reusable operations that only perform an action, while `void'(expression)` is useful when a returned value is intentionally ignored.

They can appear in verification code for:

- Utility functions
- Logging operations
- Display/reporting helpers
- Explicitly discarded function results

3.11.5 Complete Runnable Code

```
module sv_029_void_functions;

    // Void function: performs an operation without returning a value
    function void print_sum(input int a, input int b);
        $display("Sum = %0d", a + b);
    endfunction

    // Normal function returning an integer
    function int multiply(input int a, input int b);
        return a * b;
    endfunction

    int result;

    initial begin
        print_sum(10, 20);

        result = multiply(5, 4);
        $display("Product = %0d", result);

        // Evaluate the function but intentionally discard its return value
        void'(multiply(7, 3));
        $display("Return value discarded using void'()");
    end

endmodule
```

3.11.6 Output

```
Sum = 30
Product = 20
Return value discarded using void'()
```

3.12 SV-030 — Packages

A **package** groups related SystemVerilog declarations into a common namespace.

Packages can contain items such as:

- Parameters
- Typedefs
- Functions
- Tasks
- Classes

They are especially useful in verification because common definitions can be shared across multiple testbench components.

3.12.1 `package`

A package is declared using:

```
package common_pkg;

    parameter int DATA_WIDTH = 8;

    typedef logic [DATA_WIDTH-1:0] data_t;

endpackage
```

Everything declared inside the package belongs to its package scope.

3.12.2 `import`

The package can be imported using:

```
import common_pkg::*;
```

The `*` imports the package members so they can be referenced directly.

For example:

```
data_t data;
```

instead of explicitly qualifying the type.

3.12.3 Scope Resolution `::`

The scope resolution operator accesses a package member explicitly:

```
common_pkg::DATA_WIDTH
```

This is useful when you want to clearly identify where a declaration comes from or avoid namespace conflicts.

A package member can therefore be accessed either through an import:

```
DATA_WIDTH
```

or explicitly:

```
common_pkg::DATA_WIDTH
```

3.12.4 Verification Relevance

Packages are fundamental in larger verification environments because they allow common definitions to be shared between:

- Testbenches
- Transactions
- Interfaces
- Drivers
- Monitors
- Scoreboards
- UVM components

For example, transaction types and common parameters can be declared once in a package and reused throughout the verification environment.

3.12.5 Complete Runnable Code

```
package common_pkg;

    parameter int DATA_WIDTH = 8;

    typedef logic [DATA_WIDTH-1:0] data_t;

    function automatic int add_numbers(input int a, input int b);
        return a + b;
    endfunction

endpackage

module sv_030_packages;

    // Import package members into this scope
    import common_pkg::*;

    data_t data;
    int result;

    initial begin
        data = 8'hA5;
        // Access imported members directly
        result = add_numbers(10, 20);

        $display("DATA_WIDTH = %0d", DATA_WIDTH);
        $display("data       = %h", data);
        $display("sum        = %0d", result);
        // Explicit package access using scope resolution
        $display("Package DATA_WIDTH = %0d", common_pkg::DATA_WIDTH);
    end

endmodule
```

3.12.6 Output

```
DATA_WIDTH = 8
data       = a5
sum        = 30
Package DATA_WIDTH = 8
```

3.13 SV-031 — Package + Types + Classes

A SystemVerilog package can group reusable **typedefs, enums, classes, and functions** into one namespace.

This structure is especially useful in verification environments where common transaction types, states, classes, and utility functions need to be shared across multiple components.

3.13.1 Typedefs in a Package

A custom data type can be defined inside the package:

```
typedef logic [7:0] data_t;
```

This allows `data_t` to be reused wherever the package is imported.

3.13.2 Enums in a Package

An enumeration can represent defined transaction or protocol states:

```
typedef enum {
    IDLE,
    WRITE,
    READ,
    DONE
} operation_t;
```

The enum type can then be used by classes or other testbench components.

3.13.3 Classes in a Package

Classes can also be declared inside a package:

```
class transaction;

    data_t data;
    operation_t operation;

    function new(data_t data, operation_t operation);
        this.data = data;
        this.operation = operation;
    endfunction

endclass
```

The class can use types defined earlier in the same package.

3.13.4 Package Functions

Reusable utility functions can be placed inside the package:

```
function automatic int add_numbers(input int a, input int b);
    return a + b;
endfunction
```

After importing the package:

```
import verification_pkg::*;
```

the package members can be used directly.

3.13.5 Verification Relevance

This organization is commonly used to share:

- Transaction data types
- Protocol enums
- Verification classes
- Utility functions
- Common parameters and typedefs

It provides a foundation for larger non-UVM and UVM verification environments.

3.13.6 Complete Runnable Code

```
package verification_pkg;

    typedef logic [7:0] data_t;

    typedef enum {
        IDLE,
        WRITE,
        READ,
        DONE
    } operation_t;

    class transaction;

        data_t data;
        operation_t operation;

        function new(data_t data, operation_t operation);
            this.data = data;
            this.operation = operation;
        endfunction

        function void display();
            $display("Transaction: operation=%s, data=%h",
                     operation.name(), data);
        endfunction

    endclass

    function automatic int add_numbers(input int a, input int b);
        return a + b;
    endfunction

```

```
endpackage


module sv_031_package_types_classes;

    import verification_pkg::*;

    transaction trans;
    int result;

    initial begin

        trans = new(8'hA5, WRITE);
        trans.display();

        result = add_numbers(10, 20);
        $display("Package function result = %0d", result);

    end

endmodule
```

3.13.7 Output

```
Transaction: operation=WRITE, data=a5
Package function result = 30
```